

CSE 144 Final Project Report

Transfer Learning Challenge

Group Members (max 3):

Member 1: Vaishnavi Venuturimilli (CruzID:1962506)

Spring 2026

1. Introduction

1. Problem goal and setting.

The goal of this project is to build an image classifier capable of categorizing images into 100 different classes under a set of strict data constraints. The main objective was to leverage pre-trained models to achieve a result of 60% accuracy on an unseen Kaggle test set (since there were only 10 training images per class).

2. Why transfer learning is appropriate.

Transfer learning is essential for this problem as training deep neural networks from scratch on a small set of 1,000 total images would lead to severe overfitting. By loading a model that is pre-trained on a large dataset such as ImageNet, the network can encode visuals such as edges, textures and shapes. This allows for the ability to bypass the initial feature-learning phase and focus completely on fine-tuning the final layers to map the features to the 100 specific classes.

3. Brief summary of your approach and main result.

To balance model capacity and data availability, I implemented a ResNet-50 backbone. I used a partial unfreezing strategy by locking the early layers while fine tuning layers 3 and 4. I applied a 50% dropout rate to the newly constructed classifier head. I used the AdamW optimizer with a conservative learning rate and StepLR scheduling across 20 epochs. The model successfully regularized against the small dataset and achieved a final Kaggle test accuracy of 65.45%.

2. Dataset

1. Number of classes and dataset sizes (train/val/test).

The dataset consists of 100 classes with approximately 10 samples per class. The initial training dataset was split using an 80/20 ratio through PyTorch's `random_split` (80% for the training subset and 20% for the validation subset).

2. Directory structure and label mapping details (ensure labels 0–99 match folders).

A critical preprocessing step was addressing directory sorting. I made sure that strict integer sorting was implemented, since by default the OS level lexicographical sorting can scramble the directory to label mappings. This was so that class folder '0' correctly mapped to label 0 up until 99.

3. Preprocessing (resize, normalization) and data augmentation used.

To 'expand' the limited dataset and combat the 10 image per class constraint, I utilized a targeted augmentation pipeline. I implemented `RandomHorizontalFlip` with a 50% probability and a `RandomRotation` of up to 15 degrees to prevent the network from memorizing the exact pixel locations. For both the training and validation pipelines, all images were resized to 224x224 to match the standard ResNet input dimensions. The images were then converted to tensors and standardized using the ImageNet default mean [0.485, 0.456, 0.406], and standard deviation [0.229, 0.224, 0.225].

3. Implementation

3.1 Model

1. Pretrained backbone used (e.g., ResNet/EfficientNet/ViT) and why.

I selected ResNet-50 pre-trained on ImageNet. Through ablation testing, I found that ResNet-50 was the sweet spot for parameter capacity. Because deeper models consist of too many parameters and can overfit the tiny dataset quickly, and shallower models don't have the feature depth required to distinguish between a 100 classes, I used ResNet-50. ResNet's residual skip connections provide fantastic gradient stability during fine tuning of deeper layers.

2. Architecture changes (classifier head, pooling, dropout, etc.).

The base model's default for 1000-class fully connected layer was removed. I replaced it with a custom sequential classifier head consisting of a Dropout layer ($p = 0.5$) followed by a Linear layer mapping the input features down to the 100 specific Kaggle classes. The Dropout was a necessary regularization technique to randomly penalize the network and force feature generalization.

3. Fine-Tuning Strategy

Instead of fully unfreezing the model, I used a partial and discriminative freezing strategy. The initial convolutional blocks, Layer 1 and Layer 2 were strictly frozen (`requires_grad = False`). I explicitly unfroze Layer 3 and Layer 4 alongside the new classifier head. This allowed the network to retain pre-trained texture and shape recognition while heavily adapting its deepest feature maps to this particular dataset.

3.2 Training

1. Loss function and optimizer.

I used standard Cross-Entropy Loss for this task. Cross-entropy is mathematically ideal for multi-class classification due to its log scale, which heavily penalizes the model for highly confident but incorrect predictions, rather than treating all errors linearly or equally. The AdamW optimizer was used with an initial learning rate of 1×10^{-4} and a weight decay of 0.1. AdamW was specifically selected over standard Adam because it mathematically decouples the weight decay calculation from the gradient update step. Therefore L2 regularization becomes significantly effective in combating the overfitting risk posed by the constraint of 10 images per class.

2. Learning rate, scheduler, batch size, epochs, weight decay.

Training ran for 20 epochs with a batch size of 32. I implemented a StepLR scheduler to decrease the learning rate by a factor of 0.5 every 5 epochs. This step-down approach allowed the model to make broader parameter updates in the early stages and progressively finer adjustments later. This proved to be critical for preserving the valuable pre-trained ImageNet features in Layers 3 and 4 without overwriting them. Weight decay was set to 0.1 within the AdamW optimizer as a primary regularization term (to prevent the model from memorizing the limited training data).

3. Hardware/software environment (GPU, PyTorch version).

Training was conducted locally using PyTorch version 2.12.0, utilizing the Apple Silicon mps device backend.

4. Experiments

1. Baseline setup.

I considered multiple models including ResNet-18 and ResNet-152, but ultimately chose ResNet-50 because it hit the optimal balance that was deep enough to extract rich features but not so much that it memorizes the whole dataset. For the initial baseline, I attempted a very conservative fine-tuning strategy. I completely froze the pretrained network and only unfroze Layer 4 and the custom classifier head to update. Images were then resized directly to 224x224 to match the standard input size expected by ResNet models. Without a learning rate scheduling or dropout, the baseline pipeline failed to capture enough features for the 100 classes, resulting in severe overfitting and an accuracy of approximately 30-40%. The number of epochs was set to 10 initially.

2. Hyperparameter tuning plan and validation method.

I evaluated model performance using a validation set split (20%) from training data. I tracked validation accuracy and validation loss as my primary metrics for comparing between training and validation. Hyperparameters were tuned across experiments, isolating specific regularization changes to understand their individual impact. The parameters I explored included the layer unfreezing, dropout rate, learning decay schedules, weight decay, and augmentations such as rotations and flips.

3. Ablations (augmentations, model size, freezing strategy, LR, etc.).

To improve upon the 40% baseline, ablation testing focused heavily on balancing adaptable capacity with strict regularization.

- Freezing Strategy: Unfreezing the entire network was tested but led to high overfitting. The breakthrough came from expanding the unfreezing to include both Layer 3 and Layer 4, giving the model enough flexibility to learn the classes.
- Regularization (Dropout): A 50% Dropout rate ($p=0.5$) was introduced directly before the final classification step to randomly penalize neurons and force generalization.

- Scheduler: Introducing the StepLR scheduler prevented the model from getting stuck in a minima.
- Augmentations: Random Horizontal flips ($p=0.5$) and random rotations (up to 15 degrees) were introduced to artificially expand the limited training data's variance.

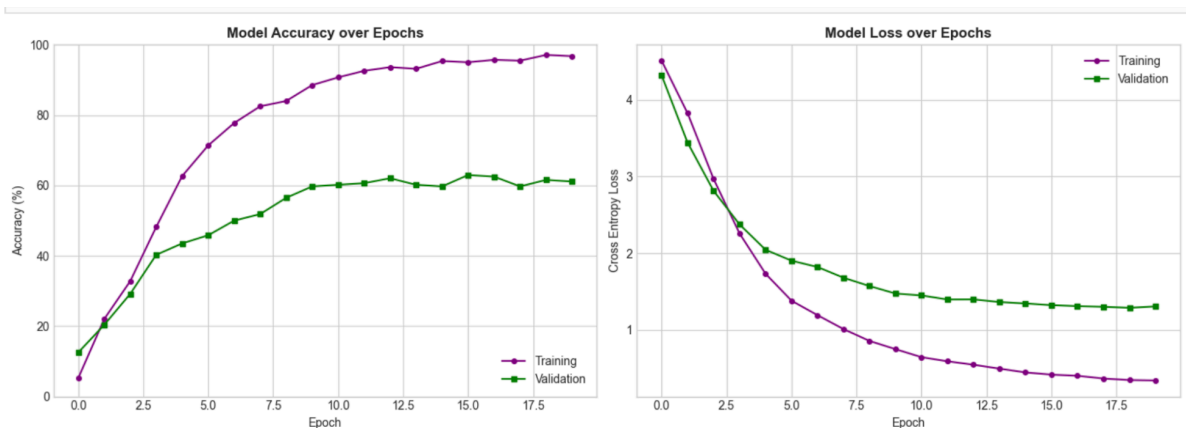
5. Results

1. Training/validation accuracy (and loss) and any key curves/plots (optional).

During the tuning process, I experimented with different parameter values to identify what worked best with my training pipeline. I observed that unfreezing both Layers 3 and 4 combined with a StepLR scheduler was the step that pushed the 40% baseline accuracy to over the 60% mark.

From my best and final run, the training accuracy climbed towards 100% which is expected of the 10-image constraint. However, due to the heavy regularization (0.5 Dropout and 0.1 Weight Decay), the validation accuracy stabilized and plateaued in the 61%-62% range. It was successfully able to avoid the spike in validation loss.

The curves show a distinct wall near epoch 10, where the model successfully extracted all generalized features and validation accuracy flattened while the training accuracy continued to climb as the model memorized the training distribution. These trends are depicted in the graphs below.



2. Kaggle public leaderboard score (and submission settings).

✓	submission.csv Complete · 8d ago	0.65454	☐
✓	submission.csv Complete · 8d ago	0.59090	☐
✓	submission.csv Complete · 8d ago	0.17272	☐

46	VV		0.65454	4	35m
----	----	---	---------	---	-----

3.

As seen in the screenshot of the Kaggle leaderboard above, my highest submitted score is ~65%.

6. Discussion

1. What worked best and why.

The most significant decision that contributed to the final score was the targeted partial unfreezing strategy paired with heavy regularization. Unlike earlier pipelines that either underfit by only unfreezing one layer (Layer 4), or extreme overfitting by unfreezing the entire network, unfreezing both layers 3 and 4 provided the exact sweet spot of model capacity. To counteract the parameter adaptability on this dataset with only 10 images per class, the dropout rate of 0.5 and a high weight decay of 0.1 within the AdamW optimizer were essential. These settings forced the network to learn generalized features rather than memorizing. Implementing the StepLR scheduler decaying the learning rate by 0.5 every 5 epochs proved to critically improve the model performance without destroying the large gradient changes.

2. Failure cases, overfitting/underfitting observations.

I observed significant underfitting in the initial baseline. By completely freezing the pre-trained ResNet-50 backbone and only training the new classification head, the model lacked the capacity to learn the features plateauing at ~40% validation accuracy. On the other hand, when attempting to unfreeze the entire model, I encountered rapid overfitting; training accuracy spiked to nearly 100% within the first few epochs while validation accuracy plummeted, showcasing the network explicitly memorizing the training noise. In the final

optimized run, I still observed the expected data wall around epoch 10, training accuracy continued its climb toward 100% while validation accuracy stabilized in the low 60% range. These results still do indicate that the model eventually exhausted the available generalized features. But it can be concluded that heavy regularization successfully prevented the violent spike in validation loss that typically accompanies severe overfitting.

3. Limitations and concrete next improvements.

The main limitation of this project is the extreme data scarcity. Even with transfer learning and foundational data augmentation (rotations and flips), the 10-image-per-class strains deep learning models. The gap between the near-perfect training accuracy and the ~65% validation accuracy highlights the ongoing struggle to fully bridge the distribution gap.

In the future, I would implement advanced data augmentation techniques such as MixUp or CutMix to synthetically blend classes and force the model to learn highly localized features rather than relying on background context. Along with this, I would work on ensembling. Possibly, pairing both ResNet-50 with a Vision Transformer (ViT) backbone would hopefully help the system extract both local convolutional features and global spatial relationships pushing that 65% to a higher testing accuracy.

7. Reproducibility

1. Random seeds and determinism settings.

To make sure that all results are completely reproducible, I explicitly set seed(42) across PyTorch, Numpy, and Python's random module at the very beginning of the training pipeline. This locks in the 80/20 random_split of the training dataset, weight initialization for the custom linear classification head, and the random sequences utilized during the data augmentation phase (horizontal flips and rotations). This guarantees that the exact same data split and initialization states are used across different runs.

2. Package versions / environment setup steps.

The training and inference pipelines were developed using Python and rely on several standard data science and deep learning libraries. The primary dependencies include PyTorch (for building and training the network), Torchvision (for loading the pre-trained ResNet-50 weights and applying image transformations), Pandas (for generating the

final CSV output), PIL (for image loading), tqdm (for progress tracking), and Matplotlib (for generating performance plots).

Training was conducted utilizing PyTorch's mps (Metal Performance Shaders) backend, which allows for highly efficient, GPU-accelerated tensor operations on Apple Silicon architecture. While the code is currently hardcoded to `device = torch.device("mps")` for my local environment, it can easily be adjusted to "cuda" or "cpu" depending on the grader's available hardware.

3. Exact commands to train and to generate submission.csv.

To reproduce these results, one could simply execute the cells in the provided .ipynb Jupyter Notebook in order. The script assumes the Kaggle dataset has been unzipped into the same root directory as the notebook and expects the relative paths `train/` and `test/` to contain the images.

Running the training cell will automatically split the dataset, apply the augmentations, initialize the partially-unfrozen ResNet-50 model, execute the 20-epoch training loop, and save the resulting weights locally as `vaishnavi_cse144_finalmodel.pth`.

Running the following inference cell will load these saved weights, switch the model to `.eval()` mode. Then it will process all images in the `test/` directory and translate the PyTorch class indices back to the corresponding Kaggle folder names, and output the properly formatted `submission.csv` file directly into the directory.

8. Team Contributions

1. **Member 1: Vaishnavi Venuturimilli:** I handled 100% of the project.

9. References

- <https://pytorch.org/vision/stable/models.html>
- https://pytorch.org/tutorials/beginner/transfer_learning_tutorial.html
- <https://docs.pytorch.org/docs/2.12/optim.html>
- <https://docs.pytorch.org/vision/stable/models.html>
- CSE144 HW2
- CSE144 Lecture Slides